Lecture 4: GPUs

Building Large Foundation Models from Scratch · WS 2026/27

Asst. Prof. Dr. Martin Pawelczyk

Faculty of Computer Science, University of Vienna

Tue 13 Oct 2026

Where we are

Admin, 2 minutes

- Do we all have access to Moodle now?

Recap, 18 minutes

- Any questions?

Today

1. Anatomy of a GPU
2. Where a training step spends its time
3. FlashAttention
4. Profiling, benchmarking, and Colab
5. Wrap-up

Anatomy of a GPU

1. Anatomy of a GPU
2. Where a training step spends its time
3. FlashAttention
4. Profiling, benchmarking, and Colab
5. Wrap-up

A throughput machine

	Server CPU (64 cores)	H100 SXM
cores	64, out of order, big caches	132 SMs $\times$ 128 lanes, in order
threads in flight	128	up to $270{,}000$
fp32 peak	~ 4 TFLOP/s	67 TFLOP/s
bf16 matmul peak	~ 8 TFLOP/s	989 TFLOP/s (tensor cores)
memory bandwidth	0.3 TB/s	3.35 TB/s
on-chip fast memory	300 MB cache	50 MB L2 + 30 MB SRAM

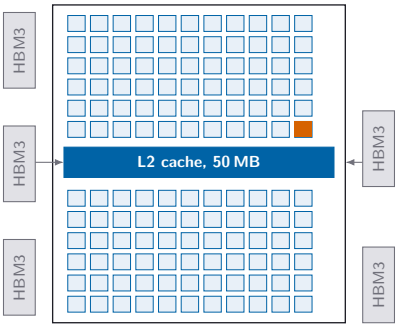
- CPU makes one thread fast: branch prediction, caches, out-of-order execution; i.e., CPU minimizes latency for a single task
- GPU keeps thousands of threads waiting so that some are always ready; i.e., GPU maximizes aggregate throughput across thousands of tasks

Key idea

Latency (i.e., time it taks for data to travel from one point to another) is hidden by parallelism. A kernel that does not expose enough parallel work leaves the machine idle, however few FLOPs it needs.

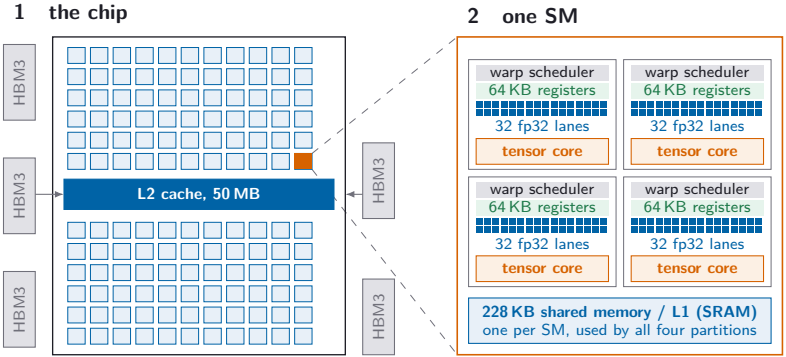
Inside an H100: one small machine, built 132 times

1 the chip



132 SMs (streaming multiprocessors), each running its own thread blocks, around one L2 that all of them share. Five HBM3 stacks: 80 GB at 3.35 TB/s.

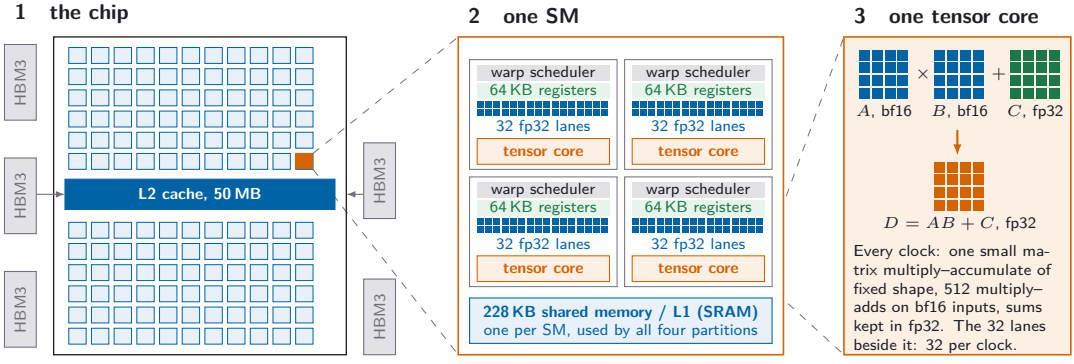
Inside an H100: one small machine, built 132 times



132 SMs (streaming multiprocessors), each running its own thread blocks, around one L2 that all of them share. Five HBM3 stacks: 80 GB at 3.35 TB/s.

Four partitions. Each: a scheduler that issues one ready warp per clock, registers, 32 fp32 lanes ("CUDA cores"), one tensor core.

Inside an H100: one small machine, built 132 times



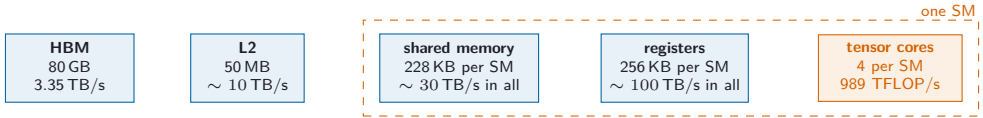
132 SMs (streaming multiprocessors), each running its own thread blocks, around one L2 that all of them share. Five HBM3 stacks: 80 GB at 3.35 TB/s.

Four partitions. Each: a scheduler that issues one ready warp per clock, registers, 32 fp32 lanes ("CUDA cores"), one tensor core.

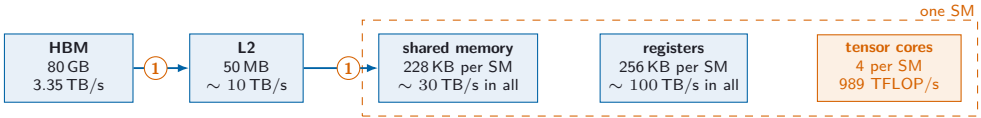
Not programmable: one instruction, one shape, one job.

Adding up: $132 \times 4 \times 32 = 16{,}896$ fp32 lanes $\rightarrow$ 67 TFLOP/s · $132 \times 4 = 528$ tensor cores $\rightarrow$ 989 TFLOP/s · $132 \times 228\,\text{KB} = 30\,\text{MB}$ SRAM · 50 MB L2 · 80 GB HBM

How it works: a matmul tile travels through the machine

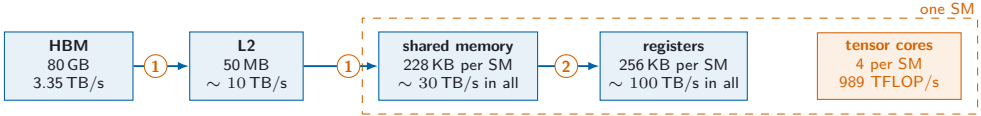


How it works: a matmul tile travels through the machine



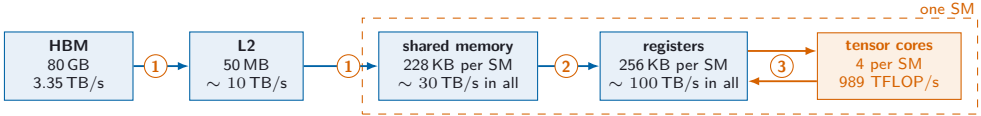
1. A thread block copies a 128×64 tile of A and a 64×128 tile of B (32 KB in bf16) into shared memory. Contiguous addresses: each warp loads in a few wide transactions.

How it works: a matmul tile travels through the machine



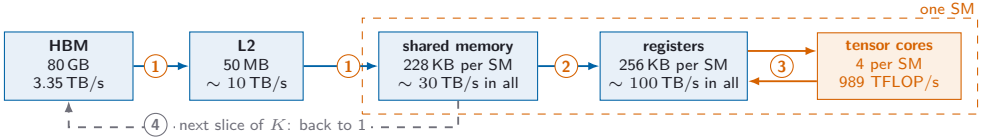
1. A thread block copies a 128×64 tile of A and a 64×128 tile of B (32 KB in bf16) into shared memory. Contiguous addresses: each warp loads in a few wide transactions.
2. Warps move 16×16 fragments of the tiles into their registers.

How it works: a matmul tile travels through the machine



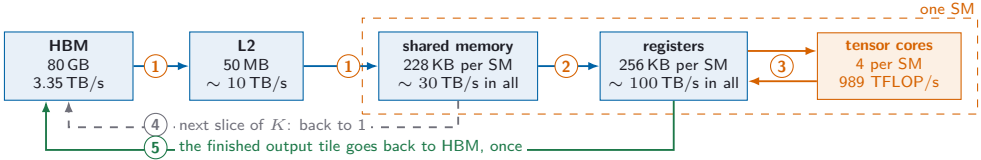
1. A thread block copies a 128×64 tile of A and a 64×128 tile of B (32 KB in bf16) into shared memory. Contiguous addresses: each warp loads in a few wide transactions.
2. Warps move 16×16 fragments of the tiles into their registers.
3. `mma`: the tensor core multiplies one fragment pair per instruction; the 128×128 fp32 sums stay in registers.

How it works: a matmul tile travels through the machine



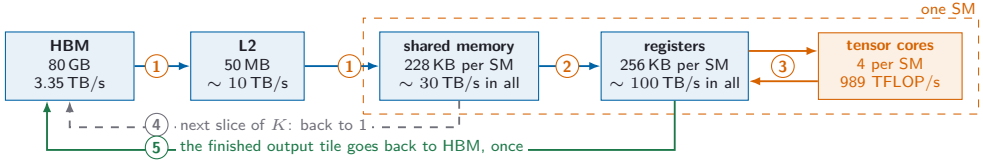
1. A thread block copies a 128×64 tile of A and a 64×128 tile of B (32 KB in bf16) into shared memory. Contiguous addresses: each warp loads in a few wide transactions.
2. Warps move 16×16 fragments of the tiles into their registers.
3. `mma`: the tensor core multiplies one fragment pair per instruction; the 128×128 fp32 sums stay in registers.
4. Next slice of K. Every loaded element is used 128 times before it leaves the SM: the reuse that beats the roofline.

How it works: a matmul tile travels through the machine

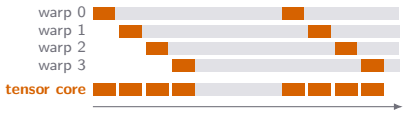


1. A thread block copies a 128×64 tile of A and a 64×128 tile of B (32 KB in bf16) into shared memory. Contiguous addresses: each warp loads in a few wide transactions.
2. Warps move 16×16 fragments of the tiles into their registers.
3. `mma`: the tensor core multiplies one fragment pair per instruction; the 128×128 fp32 sums stay in registers.
4. Next slice of K. Every loaded element is used 128 times before it leaves the SM: the reuse that beats the roofline.
5. After the whole K loop the output tile is written once.

How it works: a matmul tile travels through the machine



1. A thread block copies a 128×64 tile of A and a 64×128 tile of B (32 KB in bf16) into shared memory. Contiguous addresses: each warp loads in a few wide transactions.
2. Warps move 16×16 fragments of the tiles into their registers.
3. `mma`: the tensor core multiplies one fragment pair per instruction; the 128×128 fp32 sums stay in registers.
4. Next slice of K. Every loaded element is used 128 times before it leaves the SM: the reuse that beats the roofline.
5. After the whole K loop the output tile is written once.

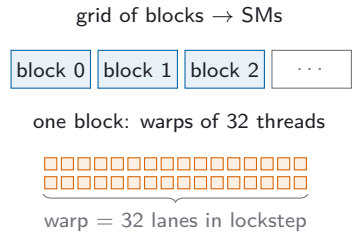


Orange: a warp computes. Grey: it waits ~ 600 cycles for a load. The scheduler issues whichever warp is ready; with enough warps the tensor core never idles.

What the machine wants: large contiguous tiles, loaded once and reused from SRAM; many warps in flight; bf16 matmuls on the tensor cores.

The execution model: kernels, blocks, warps

- A **kernel** is one function launched over a **grid** of thread **blocks**; every thread runs the same code on different indices (SIMT).
- A block (up to 1024 threads) runs on one SM and shares that SM's fast memory; blocks are independent and scheduled in any order.
- Threads execute in **warps** of 32 in lockstep; a branch that splits a warp runs both paths. A warp's loads are **coalesced** into few transactions if the addresses are contiguous.
- Each SM holds up to 64 warps; while one waits on memory, another runs. **Occupancy** is the fraction of that capacity a kernel uses.



Takeaway

A kernel is a grid of independent tiles; each tile, a few warps reading contiguous memory.

The memory hierarchy

Level (H100)	Scope	Size	Bandwidth	Latency
registers	per thread	256 KB per SM	~ 100 TB/s	1 cycle
shared memory / L1 (SRAM)	per block, on the SM	228 KB per SM, 30 MB total	~ 30 TB/s	~ 30 cycles
L2 cache	whole chip	50 MB	~ 10 TB/s	~ 200 cycles
HBM3	whole chip	80 GB	3.35 TB/s	~ 600 cycles
host RAM over PCIe 5	other side of the bus	TBs	64 GB/s	μs
other GPUs over NVLink 4	node	8×80 GB	900 GB/s	μs

- Each level is $\sim 10\times$ faster and $\sim 100\times$ smaller than the one below. HBM bandwidth is the roofline number; the levels above it are how a kernel escapes the roofline by **reuse**.
- A matmul tile in SRAM is used hundreds of times; an elementwise op uses each byte once. That is the difference between 70 % and 1 % of peak.

Takeaway

Fast memory is small and programmer-managed. Kernels are plans for moving data through this table with as few HBM trips as possible.

Tensor cores

- A tensor core computes a small dense matmul-accumulate per clock, e.g. $16 \times 16 \times 16$ in bf16 with fp32 accumulation. Four per SM, 528 per H100.
- Throughput: 989 TFLOP/s bf16 dense, 1979 fp8. The fp32 CUDA cores manage 67. Everything that is not a tensor-core matmul runs on hardware $15\times$ slower.
- Inputs: fp16, bf16, tf32, fp8 (Hopper), fp4 (Blackwell); accumulation stays in fp32. The $2\times$ "with sparsity" numbers assume 2:4 sparse weights; nobody trains that way.

Takeaway

The design goal of a model and of a kernel is the same: make as much of the work as possible a large dense matmul in a low-precision format, and keep the tensor cores fed.

`allow_tf32 = True` (in `torch.backends.cuda.matmul`) lets fp32 matmuls use tensor cores at tf32 precision: $8\times$ faster, 10^{-3} relative error. Fine for training, not for your tests?!

The CPU runs ahead

- Launches are asynchronous: Python queues a kernel and continues; the GPU works through the queue.
- Launching costs 5–$10\,\mu$s of CPU time per kernel in eager PyTorch. A small model's step launches hundreds; if each runs under $10\,\mu$s the GPU waits for the CPU: **launch-bound**.
- Anything that needs a value on the CPU synchronises: `loss.item()`, `print`, `if tensor > 0`, `.cpu()`. Do them rarely; time only with `torch.cuda.synchronize()`.



launch-bound: the GPU (orange) idles while the CPU (blue) launches the next kernel

Takeaway

Two cures: fewer, bigger kernels (fusion, larger batch), or launching without the CPU (CUDA graphs, which `torch.compile` can do).

Beyond NVIDIA

Accelerator	bf16 dense	HBM	bandwidth	ridge point	note
NVIDIA A100 (2020)	312	80 GB	2.0 TB/s	156	Colab Pro; many university clusters
NVIDIA H100 (2022)	989	80 GB	3.35 TB/s	295	the 2023–25 workhorse
NVIDIA B200 (2025)	~ 2250	192 GB	8 TB/s	~ 280	fp4 tensor cores
AMD MI250X (2021)	383 (2 GCDs)	128 GB	3.2 TB/s	120	LUMI; ROCm
AMD MI300X (2023)	1307	192 GB	5.3 TB/s	247	ROCm; large memory
Google TPU v5p (2023)	459	95 GB	2.8 TB/s	164	systolic MXU; ICI mesh; JAX

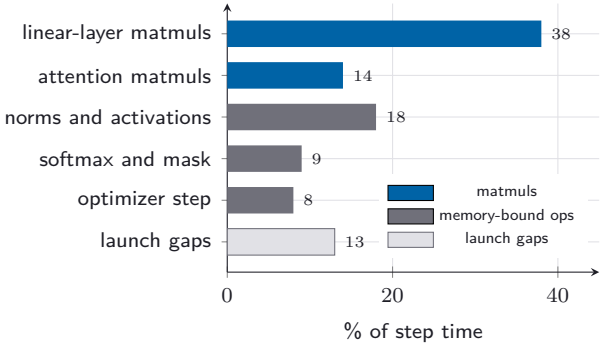
- Everything today transfers: tiles, reuse, fusion, roofline. PyTorch, Triton and `torch.compile` run on AMD via ROCm; TPUs are programmed through JAX/XLA, which fuses for you.
- Across generations FLOPs grew faster than bandwidth: the ridge point rises, more of a model becomes memory-bound.

The assignments run on Colab: T4 (free; 65 TFLOP/s fp16, no bf16), L4 or A100 (Pro).

Where a training step spends its time

1. Anatomy of a GPU
2. Where a training step spends its time
3. FlashAttention
4. Profiling, benchmarking, and Colab
5. Wrap-up

Profile of one step of the A1 model (eager PyTorch, A100)



Illustrative breakdown for a 23M-parameter model, batch 32×256, fp32, eager. Your profiler will give you the real one.

- Blue: the matmuls, about half the step, running near peak. Grey: memory-bound ops, each a trip to HBM and a fusion candidate. Light grey: the GPU waiting for the next launch.
- MFU counts only matmul FLOPs: 52 % of the time at 40 % of peak is ≈ 20 % MFU.

Takeaway

Speedups come from shrinking the grey bars, not from the matmuls.

One op, one kernel, one trip to HBM

RMSNorm in eager PyTorch on x of n elements (bf16):

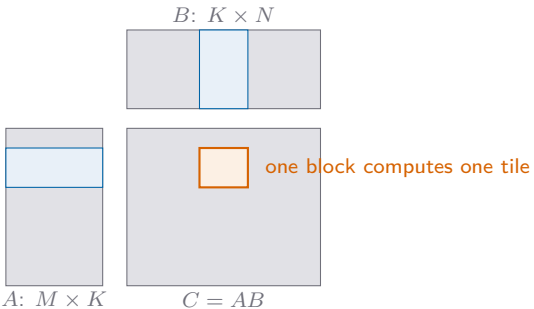
kernel	op	HBM traffic
1	`x.float()`	$2n + 4n$
2	`xf.pow(2)`	$4n + 4n$
3	`.mean(-1)`	$4n$
4	`(...+ eps).rsqrt()`	≈ 0
5	`xf * r`	$4n + 4n$
6	`.to(bf16) * g`	$4n + 2n$
6 kernels		$32n$ bytes
fused kernel	read x, write y	$4n$ bytes

- $8\times$ the traffic, 6 launches instead of 1, fp32 intermediates in memory.
- Same for softmax, residual adds, SwiGLU, RoPE, the loss, the optimizer. None of it is in $6ND$; all of it is in the profile.

Key idea

Eager PyTorch runs one kernel per op, so memory-bound ops pay the HBM round trip per op instead of per fused group.

Matmuls: tiles, shapes and waves



- Each block computes a $T_M \times T_N$ tile of C, streaming slabs of A and B through SRAM and accumulating in registers.
- FLOPs per HBM byte $\approx T_M T_N/(T_M + T_N)$: ~ 85 for a 128×256 tile; L2 reuse across neighbouring tiles lifts it over the ridge point.
- For you: dimensions that are multiples of 64 tile without waste; padding V from $50{,}257$ to $50{,}304$ speeds up the head. 132 SMs: 133 tiles take two rounds.

FlashAttention

1. Anatomy of a GPU
2. Where a training step spends its time
3. FlashAttention
4. Profiling, benchmarking, and Colab
5. Wrap-up

Online softmax: the softmax of a row, one block at a time

- Softmax needs the row maximum and the row sum: two passes before any output. FlashAttention only ever sees the current block.
- Keep three running quantities per query row: max m, normaliser ℓ, unnormalised output o. On seeing a new block of scores s with values v:

$$
\begin{aligned}
m' &= \max(m, \max s) \\
\ell' &= e^{m-m'}\,\ell + \textstyle\sum_j e^{s_j - m'} \\
o' &= e^{m-m'}\,o + \textstyle\sum_j e^{s_j - m'}\, v_j
\end{aligned}
$$

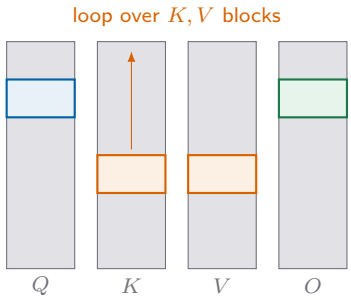
- At the end, output $= o/\ell$. Exact, in any block order [Milakov & Gimelshein, 2018].

Takeaway

The rescaling factor $e^{m-m'}$ corrects everything computed under the old maximum. Nothing about a row needs to be known in advance.

Question 2 for Tuesday: check that merging two blocks this way gives the same result as the ordinary softmax.

FlashAttention: tiling with the online softmax



one block of Q rows stays in SRAM with (m, ℓ, o); the output tile is written once

- One thread block owns a tile of Q rows. It loops over tiles of K, V, computes $s = qk^\top$ in SRAM, updates (m, ℓ, o), and never writes s or p to HBM [Dao et al., 2022].
- Traffic and memory $O(Ld)$ instead of $O(L^2)$.
- Backward: store only the per-row logsumexp; recompute p tile by tile. More FLOPs, far fewer bytes, net faster.
- Causal mask: tiles above the diagonal are skipped; half the work disappears.

FlashAttention in practice

	Year	GPU	Result
FlashAttention	2022	A100	2–4×, memory linear in L
FlashAttention-2	2023	A100	2× over v1, 50–73 % of peak
FlashAttention-3	2024	H100	75 % of peak, fp8

- In PyTorch: `F.scaled_dot_product_attention(q, k, v, is_causal=True)` dispatches to a FlashAttention or memory-efficient kernel; `enable_gqa=True` handles grouped heads; sliding windows via `flex_attention`.
- Exact attention. Nothing is approximated; the loss curves are identical up to bf16 noise.

A1

Your own `sdpa` must pass the tests. For the training runs, swap in `F.scaled_dot_product_attention` and report the difference in tokens/s and peak memory at $L = 256$ and at $L = 1024$.

Takeaway

FlashAttention did not change the model. It changed which bytes move. That is what "systems" means in this course.

Profiling, benchmarking, and Colab

1. Anatomy of a GPU
2. Where a training step spends its time
3. FlashAttention
4. Profiling, benchmarking, and Colab
5. Wrap-up

The profiler

```python
from torch.profiler import profile, ProfilerActivity
acts = [ProfilerActivity.CPU, ProfilerActivity.CUDA]
for _ in range(3): train_step()      # warm-up
with profile(activities=acts) as prof:
    for _ in range(5): train_step()
print(prof.key_averages().table(
    sort_by="cuda_time_total", row_limit=20))
prof.export_chrome_trace("trace.json")
# open trace.json at https://ui.perfetto.dev
```



a trace: gaps are launch overhead, short bars are norms and pointwise ops

- The table says *which* kernels take the time; the trace says *why*. `nvidia-smi` is not a profiler: 100 % "utilisation" means one kernel was running, not that the SMs were busy.

Takeaway

Profile before optimising, and after. Every "free win" in the checklist should move a specific bar.

Benchmarking without fooling yourself

1. Warm up: the first calls compile, allocate, and pick algorithms.
2. `torch.cuda.synchronize()` before starting and stopping the clock.
3. Repeat; report the median, and the spread.
4. Fixed shapes and dtypes, the real batch size. Batch 1 says nothing about batch 32.
5. Measure what you will optimise: tokens/s of the whole step, unless you are writing one kernel.
6. Convert to MFU (Lecture 2) or to a fraction of the roofline bound.

Classic mistakes

- Timing without synchronising (measures nothing).
- Benchmarking on a shared GPU while a colleague trains.
- Comparing to eager instead of to the roofline: "$3\times$ faster" can still be $10\times$ too slow.
- Changing two things at once.

GPU clocks vary with temperature and power; on Colab the GPU type varies between sessions. Log both.

Colab, in one slide

- Runtimes: **T4** 16 GB (free; fp16, no bf16), **L4** 24 GB and **A100** 40 GB (Pro). Which one you get varies by session; log it.
- Sessions end: idle timeouts, hard limits, disconnects. Checkpoint to Drive every few minutes.
- The VM disk is wiped: `drive.mount("/content/drive")` and keep data (`uint16` memmaps) and checkpoints there.
- Budget: the free tier covers A1 and a small A2 sweep; Pro buys faster GPUs, longer sessions.

Before your first A2 run

1. Run the Lecture 2 matmul benchmark; record GPU type and TFLOP/s.
2. Run one A1 step; compare tokens/s with the roofline.
3. Kill the runtime mid-run and resume from Drive.

T4: fp16 autocast with a `GradScaler`, or fp32. Own GPU: same code.

Wrap-up

1. Anatomy of a GPU
2. Where a training step spends its time
3. FlashAttention
4. Profiling, benchmarking, and Colab
5. Wrap-up

Summary

1. A GPU hides latency with parallelism and earns its FLOPs on tensor cores; everything else on the chip is $15\times$ slower and exists to feed them.
2. The memory hierarchy is the story: each level $10\times$ faster and $100\times$ smaller. Kernels are plans for reusing data on chip; matmuls reuse a tile hundreds of times, elementwise ops not at all.
3. Eager PyTorch launches one kernel per op, so memory-bound ops pay the HBM round trip each time and small models are launch-bound. Fusion and `torch.compile` fix most of it.
4. FlashAttention is exact attention with the $L \times L$ matrix never written: online softmax plus tiling. Bytes, not FLOPs, were the problem.

Next: Lecture 5 – Parallelism (Tue 20.10., PC room)

Coming up

- Collectives and their costs; data parallelism, ZeRO and FSDP
- Tensor, pipeline, sequence and expert parallelism
- Putting it together: how Llama 3 405B and DeepSeek-V3 were laid out

Deadline: A1 due Wednesday 21.10., 23:59, on Moodle.

Reading

- Dao et al. (2022), sections 2–3: FlashAttention
- He (2022), Making deep learning go brrr (blog)
- Triton tutorials 1–2 (vector add, fused softmax)
- Optional: Austin et al. (2025), How to scale your model, chapters 1–3

Do

- Profile your A1 model

References

- Williams, Waterman & Patterson (2009). Roofline: An insightful visual performance model for multicore architectures. *CACM*.
- NVIDIA (2025). CUDA C++ programming guide. `https://docs.nvidia.com/cuda/cuda-c-programming-guide/`.
- NVIDIA (2022). H100 Tensor Core GPU datasheet. `https://www.nvidia.com/en-us/data-center/h100/`.
- He (2022). Making deep learning go brrrr from first principles. `https://horace.io/brrr_intro.html`.
- Austin et al. (2025). How to scale your model: A systems view of LLMs on TPUs. `https://jax-ml.github.io/scaling-book/`.
- Ansel et al. (2024). PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. *ASPLOS*.
- Milakov & Gimelshein (2018). Online normalizer calculation for softmax. arXiv:1805.02867.
- Rabe & Staats (2021). Self-attention does not need $O(n^2)$ memory. arXiv:2112.05682.
- Dao et al. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *NeurIPS*.
- Dao (2023). FlashAttention-2: Faster attention with better parallelism and work partitioning. arXiv:2307.08691.
- Shah et al. (2024). FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. *NeurIPS*.

References (cont.)

- Tillet, Kung & Cox (2019). Triton: An intermediate language and compiler for tiled neural network computations. *MAPL*.
- Korthikanti et al. (2022). Reducing activation recomputation in large Transformer models. arXiv:2205.05198.
- Chowdhery et al. (2022). PaLM: Scaling language modeling with Pathways (defines model FLOPs utilisation, MFU). arXiv:2204.02311.